GILA SOFTWARE

Enterprise Platform Engineering

Architecture Decision Record

Golden Path Platform - Shared Engineering Ecosystem

Version: 0.1.0

Date: 2024-05-20

Author: Platform Engineering

Status: Approved

Classification: Internal

This document contains architectural decisions for the Golden Path Developer Experience Platform. It addresses homologation, scalability, and shift-left strategies for 10+ engineering teams.

Architecture Decision Record (ADR)

> Golden Path Platform — Shared Engineering Ecosystem
>
> Version: 0.1.0 | Date: 2024-05-20 | Author: Platform Engineering

---

1. Architecture Diagram

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                    GOLDEN PATH PLATFORM ARCHITECTURE                        │
├─────────────────────────────────────────────────────────────────────────────┤
│                                                                             │
│  ┌───────────────────────────────────────────────────────────────────────┐  │
│  │                     DEVELOPER WORKSTATION                             │  │
│  │  ┌─────────────┐   ┌─────────────┐   ┌─────────────────────────────┐  │  │
│  │  │ goldenpath  │   │  Git CLI    │   │     Local Environment       │  │  │
│  │  │    CLI      │◄─►│  + Hooks    │◄─►│  (Docker + LocalStack)      │  │  │
│  │  │  (Python)   │   │             │   │                             │  │  │
│  │  │             │   │ • pre-push  │   │ • Lambda local execution    │  │  │
│  │  │ • init      │   │ • commit-msg│   │ • DynamoDB local            │  │  │
│  │  │ • validate  │   │             │   │ • API Gateway local         │  │  │
│  │  │ • standards │   │ Enforces:   │   │ • CloudWatch logs           │  │  │
│  │  │ • dora      │   │ • Work ID   │   │                             │  │  │
│  │  │ • local env │   │ • 2-reviewers│  │ goldenpath local env        │  │  │
│  │  └─────────────┘   └──────┬──────┘   └─────────────────────────────┘  │  │
│  │                           │                                           │  │
│  └───────────────────────────┼───────────────────────────────────────────┘  │
│                             │                                               │
│                             ▼                                               │
│  ┌───────────────────────────────────────────────────────────────────────┐  │
│  │                       SERVICE REPOSITORY                              │  │
│  │  ┌─────────────┐   ┌─────────────────┐   ┌─────────────────────────┐  │  │
│  │  │ goldenpath  │   │  Application    │   │  @gila-software/        │  │  │
│  │  │   .yaml     │   │    Source       │   │  goldenpath-framework   │  │  │
│  │  │             │   │                 │   │                         │  │  │
│  │  │ project:    │   │ • Handlers      │   │ • ServiceStack          │  │  │
│  │  │   name: tx  │   │ • Tests         │   │ • PR Pipeline Gen       │  │  │
│  │  │ language:py │   │ • CDK Infra     │   │ • Integration Pipeline  │  │  │
│  │  │ work_id:FIN│   │                 │   │   Gen                   │  │  │
│  │  └─────────────┘   └─────────────────┘   └─────────────────────────┘  │  │
│  └───────────────────────────────────────────────────────────────────────┘  │
```

```
│                    │                              │
│                    ▼                              │
│  ┌────────────────────────────────────────────────────────────────┐  │
│  │              GITHUB ACTIONS (CI/CD)                       │  │
│  │  ┌───────────────┐        ┌──────────────────────────────┐  │  │
│  │  │  PR Pipeline   │        │  Integration Pipeline        │  │  │
│  │  │ (on PR open)   │        │  (on merge to main)          │  │  │
│  │  │               │        │                              │  │  │
│  │  │ 1. Detect Work ID│      │ 1. Build Artifacts           │  │  │
│  │  │ 2. Small Tests │───────►│ 2. Deploy Staging            │  │  │
│  │  │ 3. Security Scan│ (merge) │ 3. Smoke Tests              │  │  │
│  │  │ 4. Standards   │        │ 4. Deploy Production (gated) │  │  │
│  │  │ 5. Sandbox Deploy│      │ 5. DORA Telemetry Export     │  │  │
│  │  │   (optional)   │        │                              │  │  │
│  │  └───────────────┘        └──────────────────────────────┘  │  │
│  └────────────────────────────────────────────────────────────────┘  │
│                    │                              │
│                    ▼                              │
│  ┌────────────────────────────────────────────────────────────────┐  │
│  │              AWS CLOUD (Staging / Prod)                   │  │
│  │  ┌───────────┐  ┌───────────┐  ┌───────────┐  ┌─────────┐  │  │
│  │  │  Lambda   │◄─►│ API Gateway │◄─►│ DynamoDB  │◄─►│  SNS   │  │  │
│  │  │           │  │           │  │           │  │         │  │  │
│  │  │ • Python  │  │ • REST    │  │ • Tables  │  │ • Events │  │  │
│  │  │ • Go      │  │ • Routes  │  │ • PITR    │  │ • Async │  │  │
│  │  │ • TS      │  │ • Auth    │  │           │  │         │  │  │
│  │  └───────────┘  └───────────┘  └───────────┘  └─────────┘  │  │
│  │                                                  │  │
│  │  ┌──────────────────────────────────────────────────────────┐  │  │
│  │  │              CLOUDWATCH + X-RAY + DORA                │  │  │
│  │  │ • Metrics   • Logs   • Traces   • Alarms   • Audit Events │  │  │
│  │  └──────────────────────────────────────────────────────────┘  │  │
│  └────────────────────────────────────────────────────────────────┘  │
│                                           │
│  ┌────────────────────────────────────────────────────────────────┐  │
│  │              CENTRALIZED OBSERVABILITY                    │  │
│  │  ┌───────────┐  ┌───────────┐  ┌───────────┐            │  │
│  │  │  DORA     │  │  Audit    │  │ Amazon Q  │            │  │
│  │  │ Metrics   │  │  Trail    │  │ Developer │            │  │
│  │  │           │  │ (SOC 2)   │  │(AI Review)│            │  │
│  │  │ • Frequency │  │           │  │           │            │  │
│  │  │ • Lead Time │  │ • who     │  │ • PR Analysis│          │  │
```

```
│  │  │ • Failure % │   │ • what      │   │ • Security  │            │  │
│  │  │ • MTTR      │   │ • when      │   │ • Best Prac. │            │  │
│  │  │             │   │ • why       │   │             │            │  │
│  │  └─────────────┘   └─────────────┘   └─────────────┘            │  │
│  └──────────────────────────────────────────────────────────────────┘  │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

---

2. Homologation Strategy

How do we ensure 10+ teams adopt both the CLI and the Framework?

Convention over Configuration

The platform makes it easier to follow the rules than to break them:

Without Platform	With Platform
Manually configure CI/CD	generatePRPipeline() produces valid workflow
Remember branch naming	commit-msg hook auto-validates
Set up LocalStack manually	goldenpath local env --bootstrap
Inconsistent project structure	goldenpath init scaffolds standard layout
No DORA visibility	Automatic metric emission

Adoption Levers

1. Onboarding Default: New services MUST use goldenpath init
2. CI/CD Gate: GitHub Actions requires goldenpath standards check --strict
3. Git Hooks: Pre-push validation prevents non-compliant code from reaching remote
4. Metrics Visibility: Team leads see DORA dashboards; non-compliance is visible
5. Inner-Source Incentives: Teams contributing to the platform get priority support

Rollout Phases

Phase	Duration	Action	Success Criteria
Pilot	4 weeks	2 volunteer teams	Feedback + bug reports
Expansion	8 weeks	4 more teams	>80% hook installation
Standard	12 weeks	All 10+ teams	<5% standards check failures
Optimize	Ongoing	Continuous improvement	DORA metrics improve quarterly

---

3. Scalability & Bottleneck Avoidance

How does the platform team avoid becoming a bottleneck for custom pipeline features?

Extensibility by Design

The Framework uses composition over inheritance:

```
// Teams extend, they don't modify core
class PaymentStack extends ServiceStack {
constructor(scope, id, props) {
super(scope, id, props);
// Add team-specific resources
this.addPaymentProcessor();
```

Self-Service Patterns

Need	Self-Service Mechanism	Platform Touchpoint
Custom Lambda env vars	HandlerDefinition.environment	None
Additional API routes	ApiRoute[] array	None
New test step	Override testCommand	None
Custom alarm threshold	Stack props	None
New pipeline stage	Submit RFC + PR	Review only
New language runtime	Submit RFC + PR	Review + merge
Core framework bug	GitHub Issue	Platform fixes

Plugin Architecture (Future)

```
packages/
goldenpath-core/          # Stable, rarely changes
goldenpath-plugin-ml/     # ML team extension
goldenpath-plugin-data/   # Data team extension
```

Teams own their plugins. Platform owns the core contract.

Decentralized Decision Making

- Lightweight changes (< 2 files, no API change): Team self-merges
- Standard changes (new feature): 1 Trusted Committer review
- Heavyweight changes (breaking): Steering Committee

---

4. Shift-Left Strategy

How do we reduce the time between defect introduction and detection?

Validation Layers

```
┌──────────────────────────────────────────────────────────────┐
│                SHIFT-LEFT VALIDATION LAYERS                  │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  Layer 1: EDITOR / IDE                                       │
│  • goldenpath.yaml schema validation                         │
│  • Work ID linting in commit messages                        │
│  • Type checking (mypy, tsc)                                 │
│  ▼                                                           │
│  Layer 2: PRE-COMMIT                                         │
│  • commit-msg hook: Work ID enforcement                      │
│  • Formatting checks (ruff, prettier)                        │
│  ▼                                                           │
│  Layer 3: PRE-PUSH                                           │
│  • pre-push hook:                                            │
│    - Standards check                                         │
│    - Unit tests                                              │
│    - Security scan (trufflehog)                              │
│  ▼                                                           │
│  Layer 4: PR PIPELINE (GitHub Actions)                       │
│  • Small tests (fast feedback: < 5 min)                      │
│  • Property-based tests                                      │
│  • API contract validation                                   │
│  • Dependency vulnerability scan                             │
│  • Golden Path standards check                               │
│  ▼                                                           │
│  Layer 5: INTEGRATION PIPELINE                               │
│  • Full integration tests                                    │
│  • Smoke tests in staging                                    │
│  • Production deployment (gated)                             │
│  ▼                                                           │
│  Layer 6: PRODUCTION                                         │
│  • CloudWatch alarms                                         │
│  • X-Ray distributed tracing                                 │
│  • Automatic rollback on error rate threshold                │
│  • DORA failure event emission                               │
```

```
│                                          │
└──────────────────────────────────────────────────────────────────────────────┘
```

Feedback Loop Metrics

Stage	Target Time	Detection Rate
IDE/Editor	Real-time	15% of defects
Pre-commit	< 1s	10% of defects
Pre-push	< 2 min	30% of defects
PR Pipeline	< 5 min	35% of defects
Integration	< 15 min	9% of defects
Production	< 5 min MTTR	1% of defects

AI-Assisted Review (Amazon Q Developer)

Amazon Q Developer provides automated PR analysis:

- Security vulnerability detection
- Performance anti-pattern identification
- Best practice recommendations
- Code quality scoring

This acts as an additional review layer before human reviewers, catching issues that static analysis misses.

Local Environment Parity

LocalStack ensures the local environment mirrors production:

docker-compose.local.yml
services:
localstack:
image: localstack/localstack
environment:
- SERVICES=lambda,apigateway,dynamodb,sns,sqs

This eliminates the "works on my machine" class of defects detected only in staging.

Decision Log

ID	Decision	Rationale	Status

ADR-001	Python for CLI	Team expertise, rich ecosystem, uv packaging	Accepted
ADR-002	TypeScript for Framework	CDK is TS-first, type-safe workflows, pnpm	Accepted
ADR-003	uv over pip	Modern, fast, reproducible, no venv management	Accepted
ADR-004	pnpm over npm	Monorepo support, disk efficient, strict peers	Accepted
ADR-005	LocalStack for local dev	AWS parity, no cloud costs, fast feedback	Accepted
ADR-006	JSONL for DORA events	Append-only, streamable, grep-friendly	Accepted
ADR-007	OIDC for AWS auth	No long-lived secrets, GitHub-native, secure	Accepted

---

End of ADR